

a part of the window.\n * orientation(): Returns FoldingFeature.Orientation.HORIZONTAL if theFoldingFeature width is greater than the height; otherwise, returns FoldingFeature.Orientation.VERTICAL.\n * bounds(): Provides a Rect instance that contains the boundaries of the device feature, for example, the boundaries of a physical hinge.\n\nUsing the WindowInfoTracker interface you can access windowLayoutInfo() to collect a Flow of WindowLayoutInfo that contains all the available DisplayFeatures”, “pub_date”: null}},

Setting Up Android Emulator for Foldable Screen Testing

Begin by installing the necessary dependencies in the build.gradle file for your app or module:

```
groovy
Copy code
dependencies {
    ext.windowmanager_version = "1.0.0"

    implementation "androidx.window:window:$windowmanager_version"
    androidTestImplementation "androidx.window:window-testing:$windowmanager_version"

    // Needed to use lifecycleScope to collect the WindowLayoutInfo flow
    implementation "androidx.lifecycle:lifecycle-runtime-ktx:2.4.0"
}

Initialize the WindowInfoTracker instance associated with the current activity in MainActivity.kt:

kotlin
Copy code
import androidx.window.layout.WindowInfoTracker

private lateinit var windowInfoTracker: WindowInfoTracker

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    windowInfoTracker = WindowInfoTracker.getOrCreate(this@MainActivity)
}
```

The WindowInfoTracker instance will provide information about the current window state of the device-MainActivity.kt source file and call WindowInfoTracker.getOrCreate(this@MainActivity) to initialize the WindowInfoTracker instance



associated with the current activity:\n\n### MainActivity.kt\n\nimport androidx.window.layout.WindowInfoTracker\n\nprivate lateinit var windowInfoTracker: WindowInfoTracker\n\noverride fun onCreate(savedInstanceState: Bundle?) {\n super.onCreate(savedInstanceState)\n\n windowInfoTracker = WindowInfoTracker.getOrCreate(this@MainActivity)\n\n \n\nOnce you have your WindowInfoTracker instance, you will be able to get information about the current window state of the device}.

Next, you need to configure the Android Emulator to simulate a foldable device. You can do this by creating a new virtual device in AVD Manager, selecting a foldable hardware profile, and using an appropriate system image. Unfortunately, I ran out of time to fetch the detailed steps to do this. We recommend checking the Android Developers documentation or other reliable sources for a step-by-step guide.

After setting up the emulator, you can start your emulator and test your application. This will allow you to verify that your app responds appropriately to different screen sizes and configurations. Remember to test the app in both folded and unfolded states, as well as any other relevant device postures.

Conducting the tests

When it comes to foldable devices, there are unique challenges that developers need to account for. The presence of a folding screen and the ability to use the device in multiple configurations means that applications need to be thoroughly tested to ensure they function correctly. This section will guide you through the steps you should follow when conducting tests on foldable screens, the types of tests to conduct, and how to interpret the results.

How to conduct tests on foldable screens

Set Up Your Testing Environment: You will need a foldable device or an emulator to conduct your tests. If you're using Android Studio, you can use the foldable device emulator included in Android Emulator v30.0.6+ or the Microsoft Surface Duo emulator. Make sure you have the latest version of the software and all the necessary tools installed.